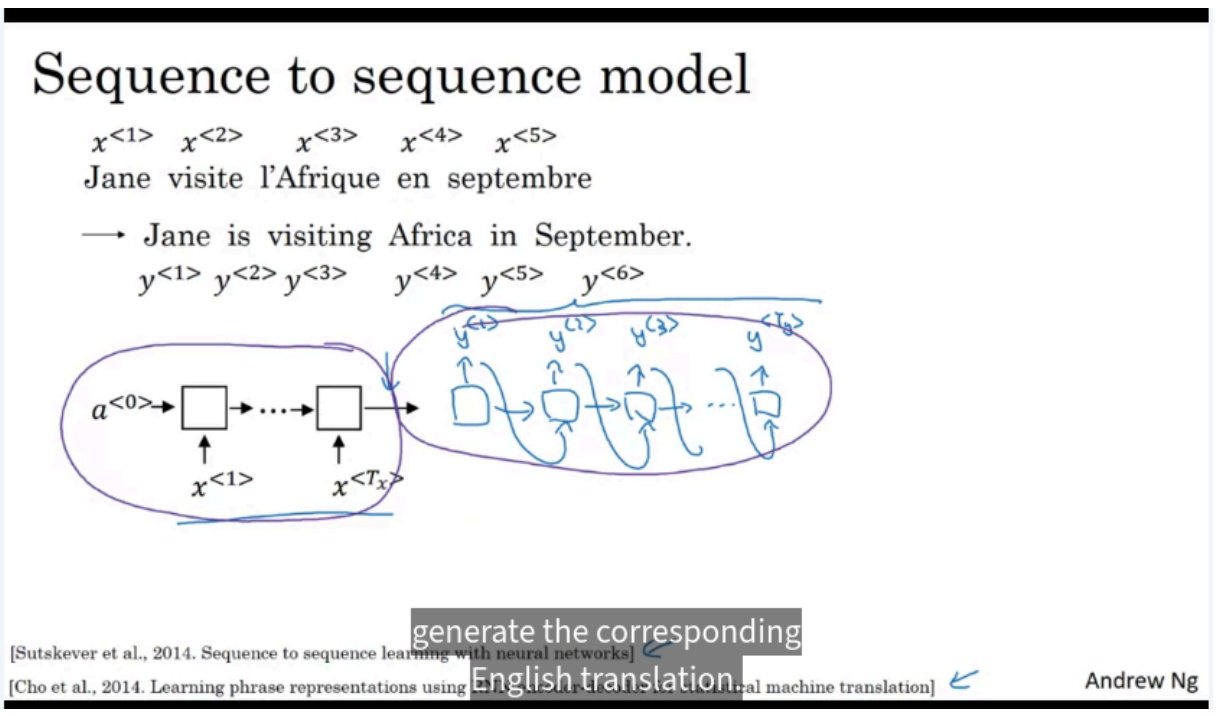
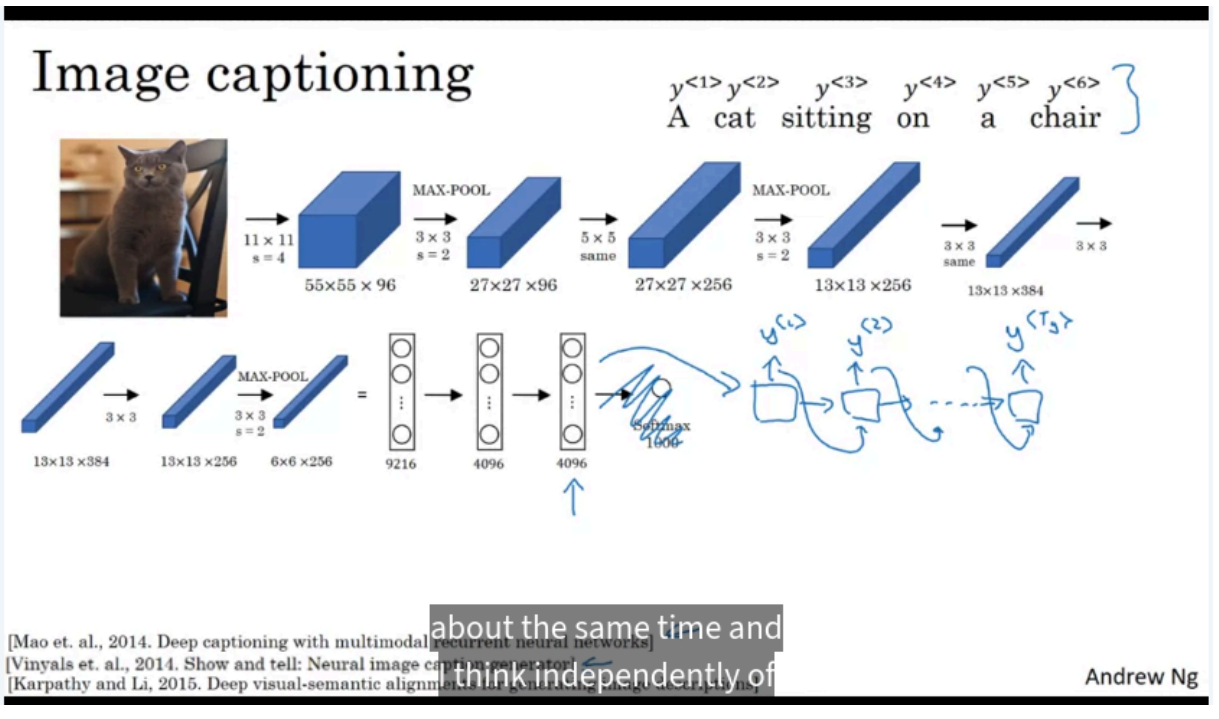
Various Sequence to Sequence Architectures

Basic Models



Seq2Seq model: For example translate machine:

- Use an encoder and a decoder



Here are the Notion-ready notes for this lesson on basic Sequence-to-Sequence models.

Sequence-to-Sequence Models

A **Sequence-to-Sequence model**, often shortened to **Seq2Seq**, is a neural architecture that maps one sequence into another sequence.

Common applications include:

- Machine translation
- Speech recognition
- Text generation
- Image captioning
- Sequence transformation tasks

The core structure is:

Input sequence → Encoder → Representation → Decoder → Output sequence

Machine Translation Example

Suppose the input is a French sentence:

> Jane visite l'Afrique en septembre

The desired English translation is:

> Jane is visiting Africa in September

Represent the input words as:

x^1, x^2, x^3, x^4, x^5

and the output words as:

$y^1, y^2, y^3, y^4, y^5, y^6$

Notice that:

$T_x \neq T_y$

The input and output sequences do not need to have the same length.

This is one of the main motivations for using an encoder-decoder architecture.

Encoder Network

The **encoder** reads the input sequence one token at a time.

It can be implemented using:

- A basic RNN
- A GRU
- An LSTM

Conceptually:

$x^1 \rightarrow x^2 \rightarrow x^3 \rightarrow \ldots \rightarrow x^{T_x}$

The encoder processes the entire input sentence and produces a vector representation:

c

This vector is intended to summarize the meaning or relevant information of the input sequence.

Conceptually:

French sentence → Encoder → c

For example:

> Jane visite l'Afrique en septembre

becomes a learned vector representation.

Decoder Network

The **decoder** receives the encoded representation c and generates the output sequence one word at a time.

Conceptually:

$c \rightarrow y^1 \rightarrow y^2 \rightarrow y^3 \rightarrow \ldots \rightarrow \text{EOS}$

For the translation example:

c

→ Jane

→ is

→ visiting

→ Africa

→ in

→ September

→ EOS

The decoder is also usually an RNN, GRU, or LSTM.

Autoregressive Decoding

The decoder generates one token at a time.

After generating one token, that token is used to help generate the next token.

Conceptually:

$\hat{y}^1$ → input for step 2

$\hat{y}^2$ → input for step 3

$\hat{y}^3$ → input for step 4

At time step t:

$P(y^t \mid y^1, y^2, \ldots, y^{t-1}, c)$

The decoder therefore predicts the next token conditioned on:

- The encoded input sentence
- Previously generated output tokens

This is an **autoregressive** process.

End-of-Sequence Token

The decoder needs to know when generation should stop.

A special token is commonly used:

EOS

which means:

> End of Sequence

The output might therefore be:

Jane → is → visiting → Africa → in → September → EOS

Once EOS is generated, decoding stops.

Basic Encoder-Decoder Architecture

The overall architecture can be summarized as:

Input sequence:

$x^1, x^2, \ldots, x^{T_x}$

↓

Encoder

↓

Context vector c

↓

Decoder

↓

Output sequence:

$y^1, y^2, \ldots, y^{T_y}$, EOS

The encoder transforms a variable-length input sequence into an internal representation.

The decoder transforms this representation into a variable-length output sequence.

Training the Seq2Seq Model

To train the model, we need paired sequences.

For machine translation:

French sentence ↔ English sentence

Example:

> Jane visite l'Afrique en septembre

paired with:

> Jane is visiting Africa in September

With enough sentence pairs, the model learns:

1. How to encode the meaning of the source sentence
2. How to generate the corresponding target sentence

The entire model can be trained end-to-end.

Why This Model Is Important

The encoder-decoder architecture was an important result because it showed that neural networks could learn to convert one variable-length sequence into another.

The model does not require:

- Hand-designed translation rules
- Explicit grammar rules
- Word-by-word correspondence

Instead, it learns the mapping from paired training examples.

Fixed-Length Context Representation

In the basic Seq2Seq architecture, the encoder compresses the entire input into one representation:

c

The decoder then generates the full output based on this vector.

So:

$x^{1}, \ldots, x^{Tx} \rightarrow c \rightarrow y^{1}, \ldots, y^{Ty}$

This works well for many cases, but it can become difficult when the input sequence is very long because the encoder must compress a lot of information into a single representation.

This limitation later motivates the **attention mechanism**.

Image Captioning as Sequence Generation

The same general architecture can also be used when the input is not a sequence.

Consider an image:

> A cat sitting on a chair

The goal is to generate a caption:

> A cat sitting on a chair.

The architecture becomes:

Image → Encoder → Feature vector → Decoder → Caption

Image Encoder

A convolutional neural network can be used to encode the image.

For example, a pretrained CNN such as AlexNet can process an image.

Normally, the final layer of a CNN performs classification:

Image → CNN → Softmax → Class

For image captioning, we can remove the final classification layer and use an intermediate feature representation.

For example, the network may produce:

$c \in \mathbb{R}^{4096}$

This 4096-dimensional vector represents visual features from the image.

So:

Image → CNN → 4096-dimensional feature vector

The CNN effectively acts as the **encoder**.

Caption Decoder

The image feature vector is then passed to an RNN-based decoder.

The decoder generates words one at a time:

Image vector

→ “a”

→ “cat”

→ “sitting”

→ “on”

→ “a”

→ “chair”

→ EOS

The process is therefore very similar to machine translation.

Machine Translation

Sequence → Encoder → Vector → Decoder → Sequence

Image Captioning

Image → CNN Encoder → Vector → Decoder → Sequence

The main difference is the type of input encoder.

General Encoder-Decoder Framework

The architecture can be generalized as:

Input modality → Encoder → Representation → Decoder → Output sequence

Possible input modalities include:

- Text
- Speech
- Images
- Video
- Sensor data

Possible output sequences include:

- Text
- Labels
- Transcriptions
- Translations

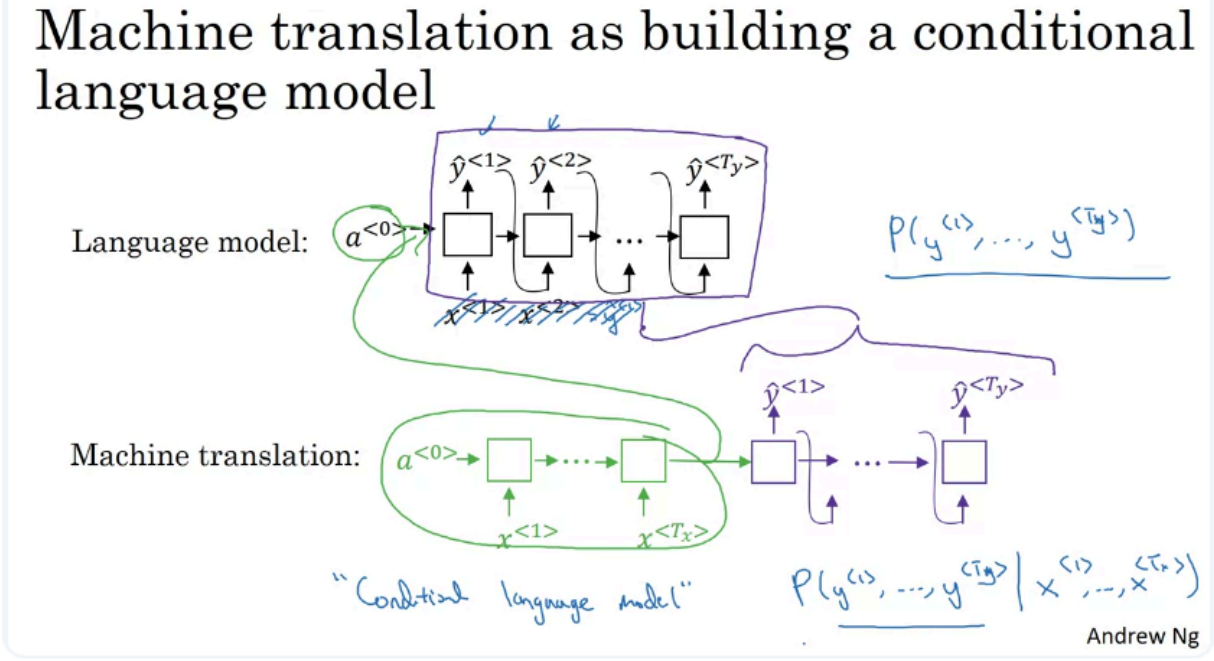
- Captions

This makes the encoder-decoder framework highly flexible.

Machine Translation vs. Image Captioning

Component	Machine Translation	Image Captioning
Input	Text sequence	Image
Encoder	RNN / GRU / LSTM	CNN
Intermediate representation	Sentence vector	Image feature vector
Decoder	RNN / GRU / LSTM	RNN / GRU / LSTM
Output	Translated sentence	Caption
Generation	Autoregressive	Autoregressive

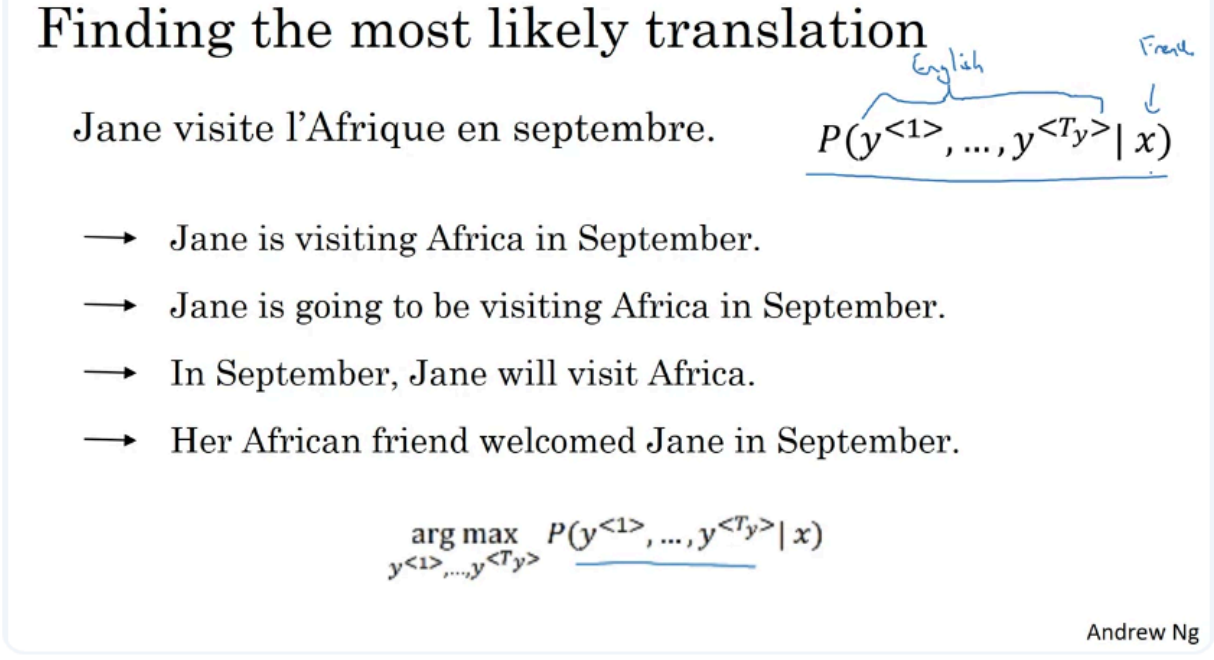
Picking the Most Likely Sentence



The green block is encoder

The purple block is decoder

The objective of this conditional language model is that they try to predict $P(y^1, ..., y^{T_y} | x^1, ..., x^{<T_x>})$



Multiple Possible Translations

For the same French input, the model may assign probabilities to many possible English sentences.

For example:

- "Jane is visiting Africa in September."
- "Jane is going to be visiting Africa in September."
- "In September, Jane will visit Africa."
- "Her African friend welcomed Jane in September."

Some translations are good, while others may be awkward or completely incorrect.

Why Random Sampling Is Not Appropriate

For generation tasks such as language modeling, we may sometimes **sample** from the model:

$$y \sim P(y|x)$$

But this is generally not what we want for machine translation.

Random sampling could produce:

- A very good translation
- An acceptable but awkward translation
- A poor translation

Instead, we want the **most probable translation**.

Machine Translation as an Optimization Problem

Given an input sentence (x), the goal is:

$$\boxed{y^* = \arg\max_y P(y|x)}$$

where:

- (x): input sentence
- (y): possible translation
- (y^*): best translation according to the model

So the problem becomes:

> Search through possible output sentences and find the one with the highest conditional probability.

Why Search Is Difficult

The number of possible output sentences is enormous.

For example, with a vocabulary of (V) words and a sentence of length (T), there are roughly: V^T

possible sequences.

Therefore, directly checking every possible sentence is computationally impossible.

We need an efficient search algorithm.

Beam Search

The most commonly used method introduced for this problem is **Beam Search**.

Its purpose is to approximately solve: $\arg\max_y P(y|x)$

without enumerating every possible output sentence.

So the overall translation process is:

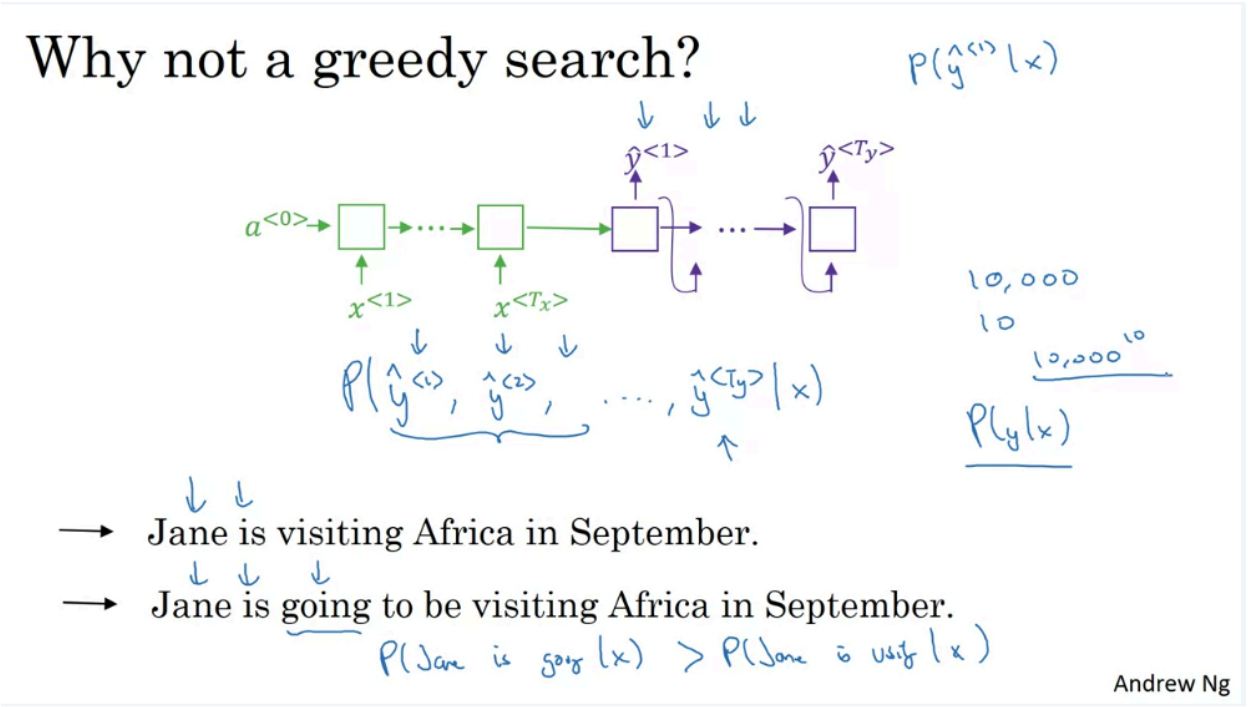
$$\boxed{\text{Input } x \rightarrow \text{Conditional Language Model } P(y|x) \rightarrow \text{Beam Search} \rightarrow \text{Best translation } y^*}$$

Key Takeaways

- Machine translation can be formulated as a **conditional language modeling problem**: $P(y|x)$
- The model assigns probabilities to many possible translations.
- We generally **do not randomly sample** translations.
- Instead, we want:

$$\boxed{y^* = \arg\max_y P(y|x)}$$

- Finding this sequence exactly is too expensive.
- **Beam Search** is used as an efficient approximate search algorithm to find a high-probability translation.



Greedy search is not always the best choice because:

- If the P(Jane is going | x) > P(Jane is visiting | x) the model will choose the term Jane is going to ... (”going” more likely appears than “visiting”)
- But “Jane is going to ...” is a longer sentence compared to “Jane is visiting”

So greedy search may work but not be appropriate

Local Optimum vs. Global Optimum

Greedy search performs:

$$\arg\max_{y_1} P(y_1|x)$$

then:

$$\arg\max_{y_2} P(y_2|y_1, x)$$

and so on.

But what we actually want is:

$$\boxed{\arg\max_{y_1,\ldots,y_T} P(y_1, \ldots, y_T|x)}$$

These are **not equivalent**.

So:

$$\text{Best next word} \not\Rightarrow \text{Best complete sentence}$$

Need for Approximate Search

We therefore have three possibilities:

Method	Problem
Greedy search	Too aggressive; may miss better sequences
Exhaustive search	Computationally impossible
Approximate search	Practical compromise

The goal of an approximate search algorithm is to find a sentence with a very high value of: $P(y \mid x)$

without evaluating every possible sentence.

Beam Search

Beam Search is a commonly used approximate search method.

Instead of keeping only **one** candidate sequence like greedy search, it keeps several promising candidates at each step.

Conceptually:

Greedy Search⇒keep 1 candidate

while:

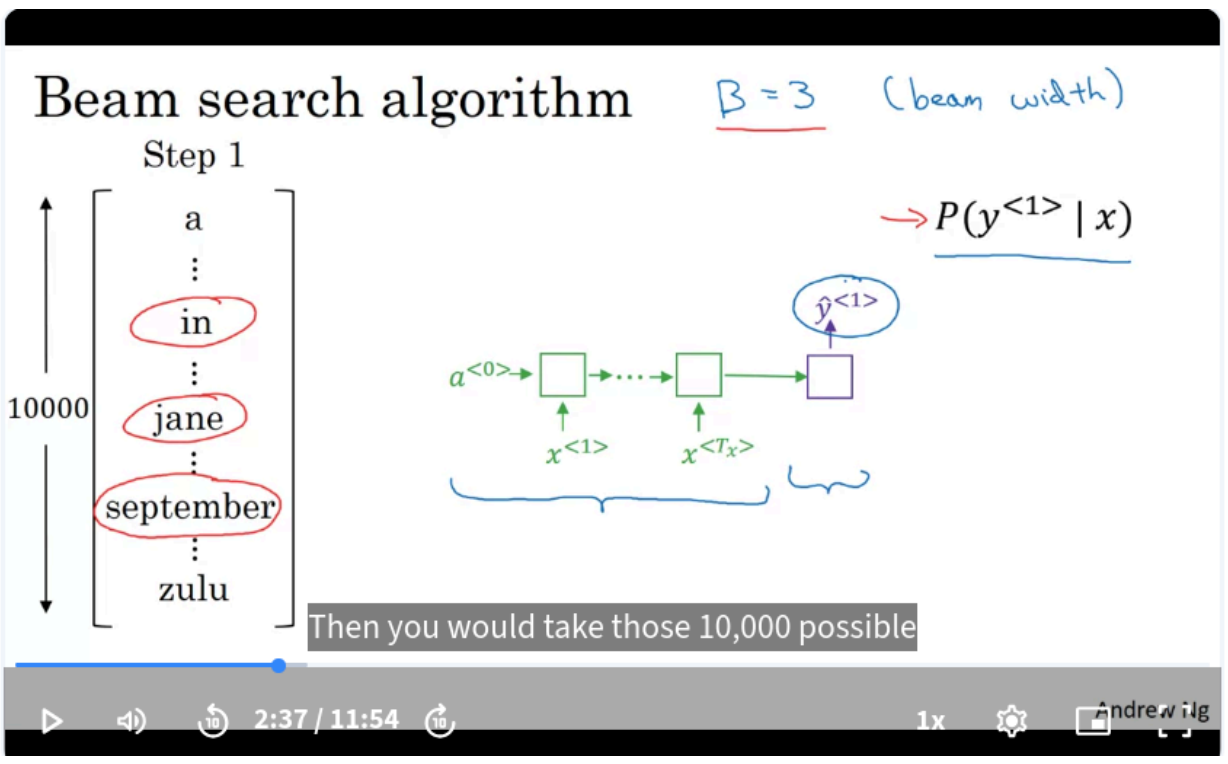
Beam Search⇒keep B candidates

where B is called the **beam width**.

This reduces the chance of making an irreversible bad decision too early.

Beam Search

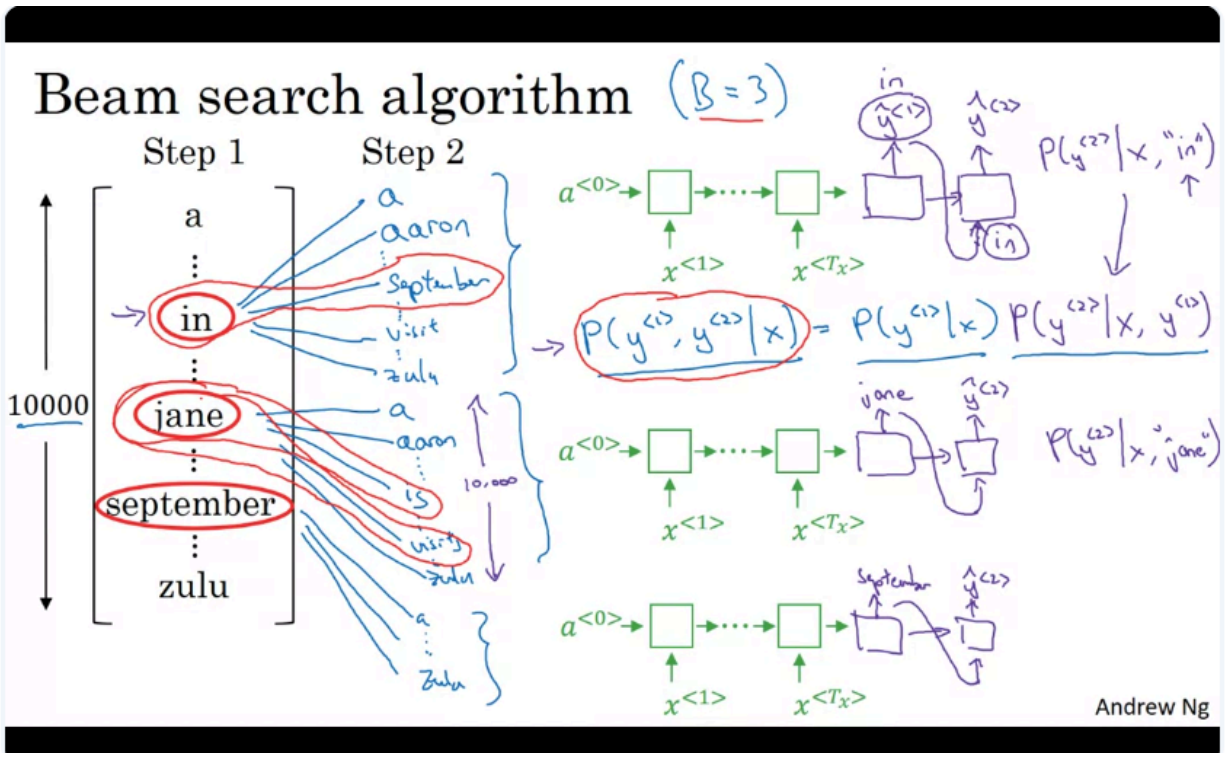
Beam Search Algorithm



Step 1: Choose the First Word

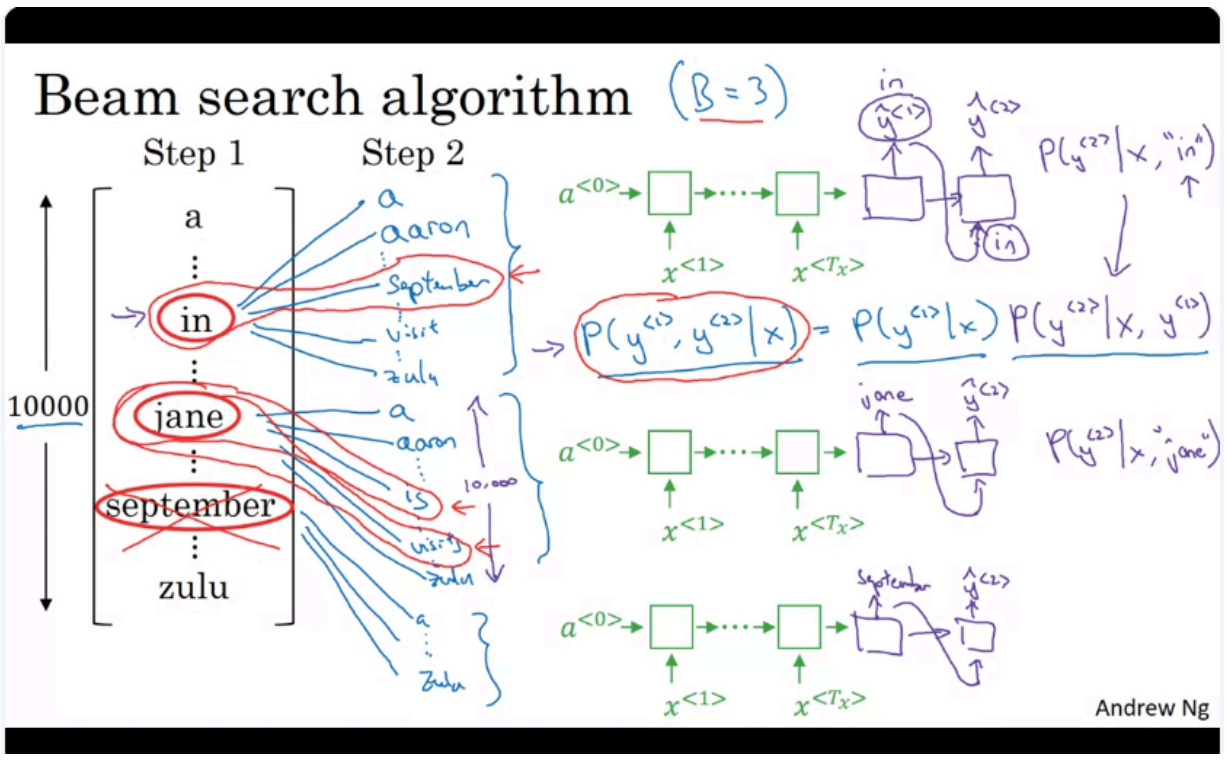
- Pass the input through the encoder
- After getting the embedding of encoder, we pass it to the first block of decoder and get the output of softmax activation function → Get the top B (Beam width) words, in this example is 3 words.

Step 2: Expand Every Candidate

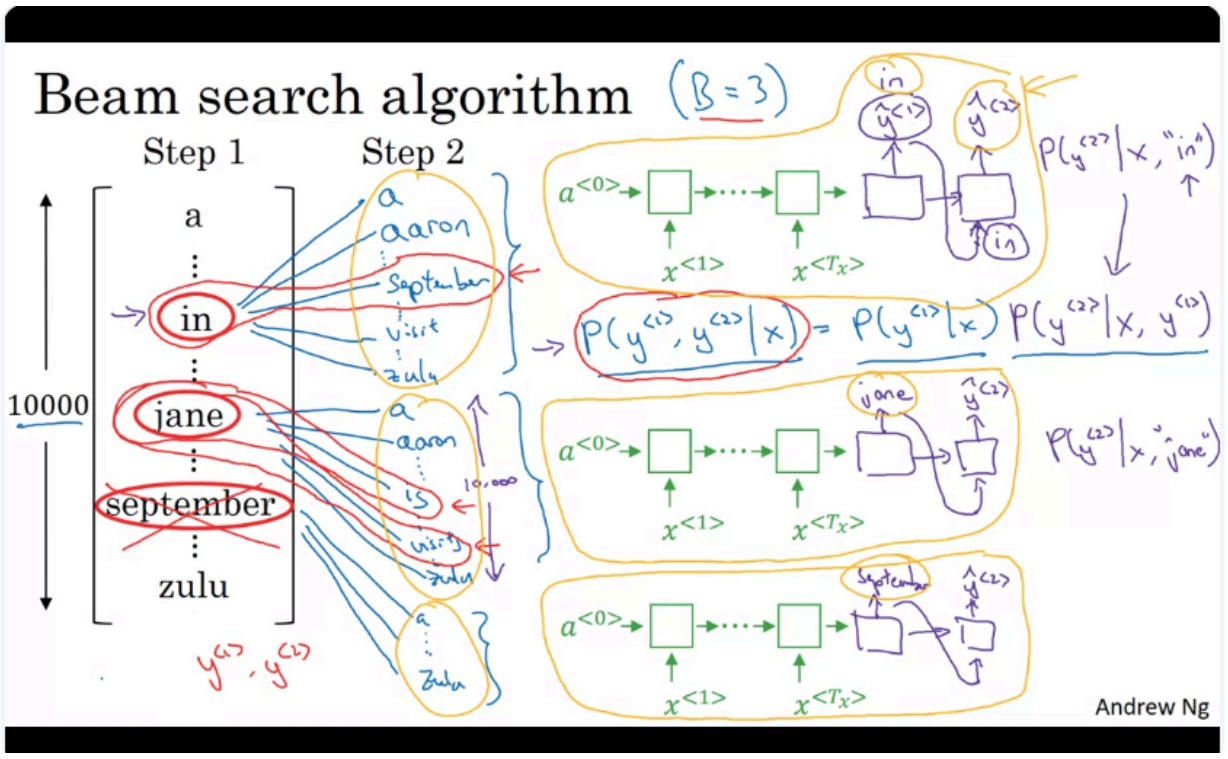


For example:

- We take 3 words: In, jane, septemper
- Then we use these three words as input of the second block of encoder and get the output of softmax. In this case, we got 3 × 10,000 = 30,000 possible cases (3 is Beam width, and 10,000 is the vocab size)
- And we get top 3 highest probability among 30,000 cases



and now because the “september” does not have the second word, so “september” can not be the first word of the translated sentence → we remove it



Efficient Neural Network Computation

With beam width:

$$B = 3$$

you conceptually maintain **three decoder states**, one for each partial sequence.

You do **not** need 30,000 separate neural networks.

Each decoder state produces a softmax distribution over all:

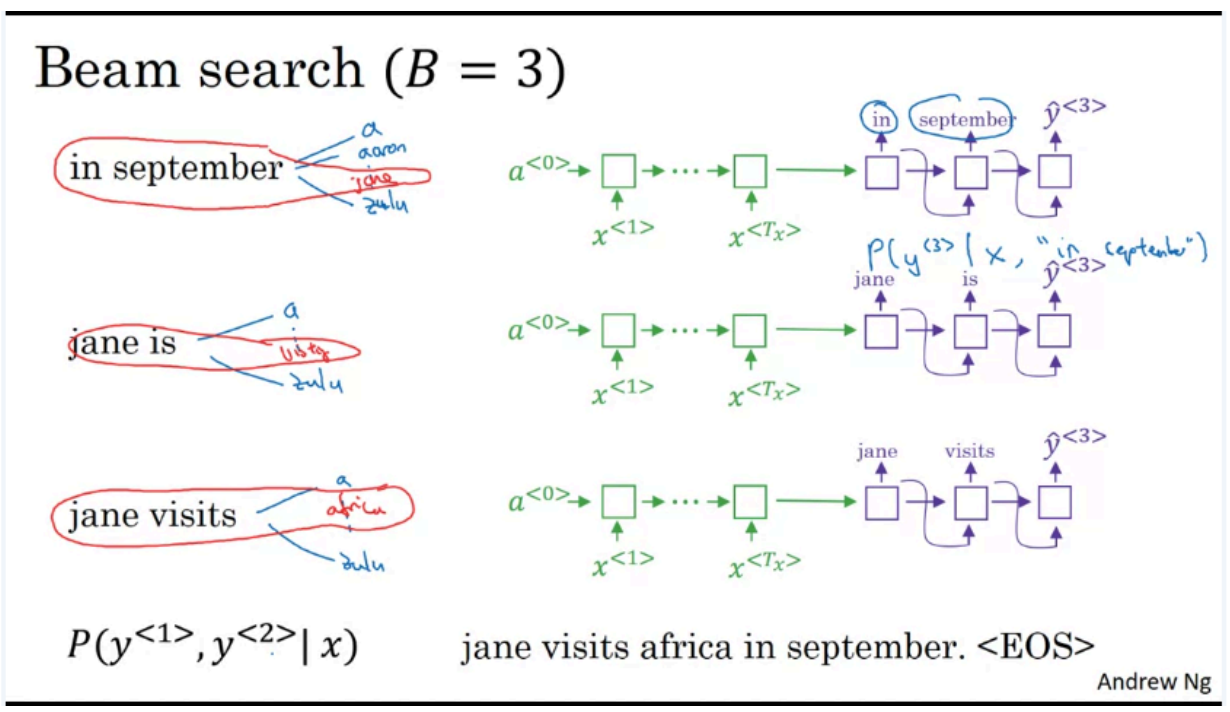
$$10,000$$

possible next words efficiently.

So the model computes roughly:

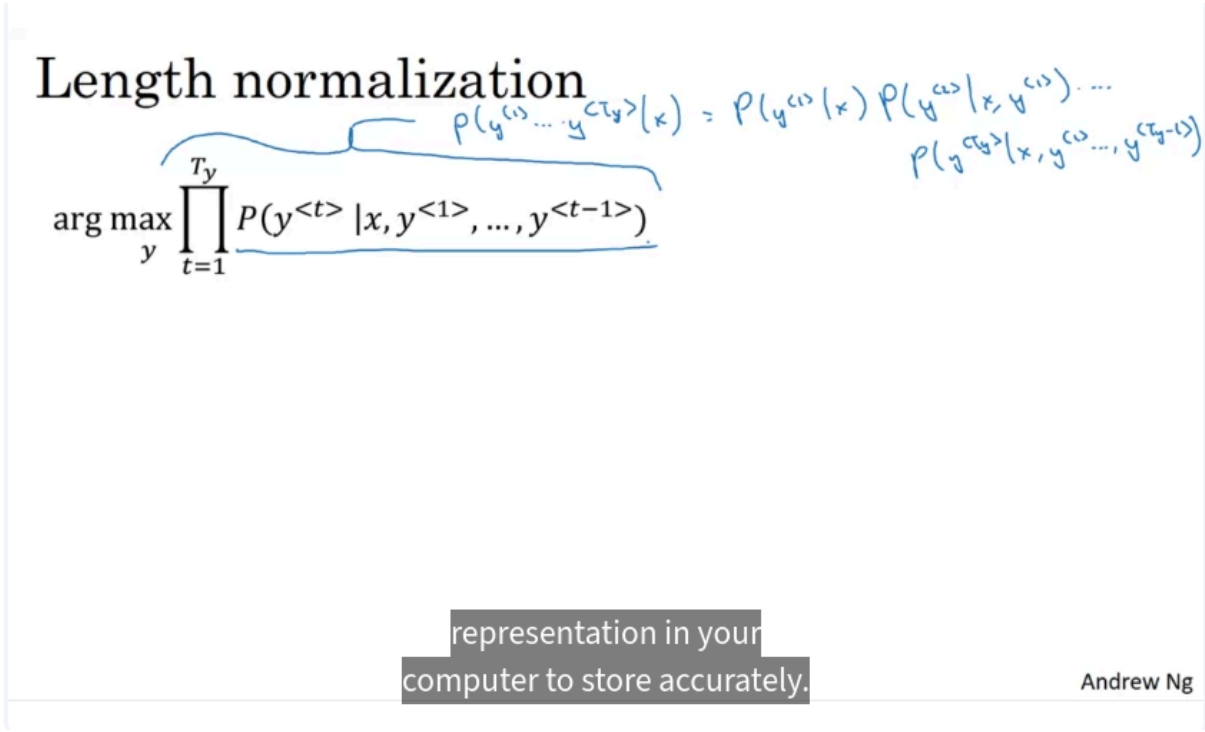
$$B$$

decoder states, each producing V possible extensions.

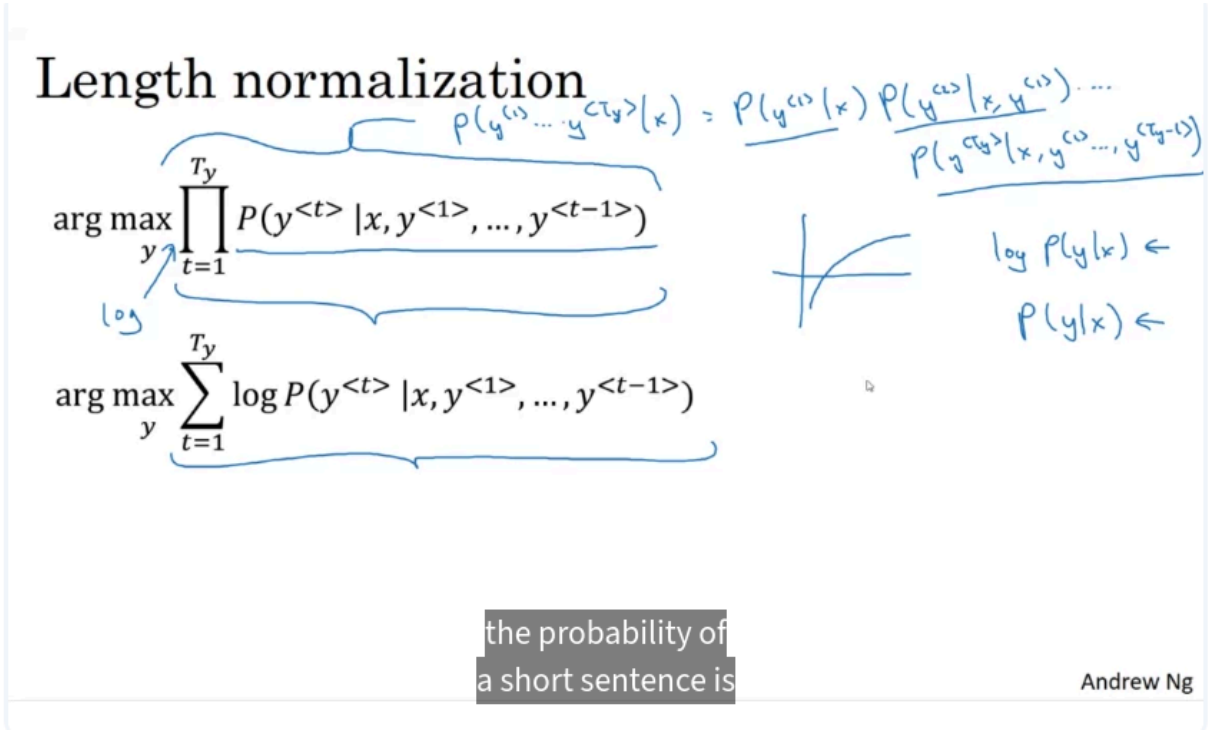


if the B is 1, the model use **greedy search**

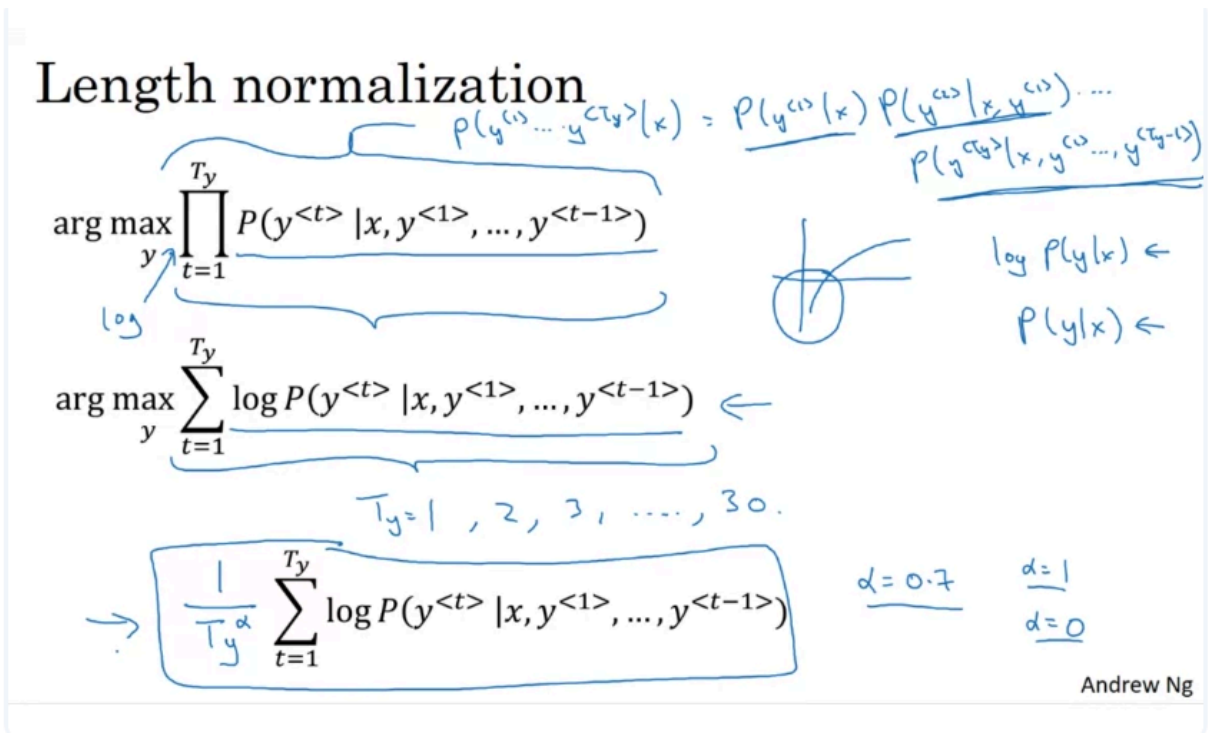
Refinements to Beam Search



→ the multiplication of many small number can lead to numerical under flow



Taking the log will make the model more stable



Length Normalization in Beam Search

- Beam Search usually scores a candidate translation using its **log probability**:

$$\log P(y|x) = \sum_{t=1}^{T_y} \log P(y_t \mid y_1, \ldots, y_{t-1}, x)$$

- Because each probability is less than 1, each log probability is typically negative.
- As a result, **longer sentences accumulate more negative terms** and can be unfairly penalized.

Why Length Normalization Is Needed

Without normalization, Beam Search may prefer shorter translations simply because they contain fewer probability terms.

For example:

$$\text{Score}(y) = \sum_{t=1}^{T_y} \log P(y_t| \cdots)$$

A longer but better translation may receive a lower score than a shorter incomplete translation.

So we want to reduce this **short-sentence bias**.

Full Length Normalization

A simple solution is to divide by the output length:

$$\boxed{\text{Score}(y) = \frac{1}{T_y} \sum_{t=1}^{T_y} \log P(y_t \mid y_{<t}, x)}$$

This computes the **average log probability per word**.

It reduces the penalty against longer translations.

Soft Length Normalization

In practice, a softer version is often used:

$$\boxed{\text{Score}(y) = \frac{1}{T_y^{\alpha}} \sum_{t=1}^{T_y} \log P(y_t \mid y_{<t}, x)}$$

where (α) is a tunable hyperparameter.

A common example is: $\alpha = 0.7$

Effect of (α)

If (α=0)

$$T_y^0 = 1$$

Therefore: $\text{Score}(y) = \sum_t \log P(y_t| \cdots)$

There is **no length normalization**.

If (α=1)

$$\text{Score}(y) = \frac{1}{T_y} \sum_{t} \log P(y_t| \cdots)$$

This is **full length normalization**.

If (0<α<1)

For example: $\alpha = 0.7$

this gives a compromise between:

- No normalization
- Full normalization

So:

$$\boxed{\alpha = 0 \rightarrow \text{no normalization}}$$

$$\boxed{\alpha = 1 \rightarrow \text{full normalization}}$$

(α) Is a Hyperparameter

- There is no strong theoretical reason that a specific value such as (0.7) must be optimal.
- It is mainly a practical heuristic.
- Different values of (\alpha) can be tested on a development set.

The goal is to find the value that produces the best translations.

Selecting the Final Translation

During Beam Search, the algorithm generates many candidate sentences of different lengths: $T_y = 1, 2, 3, \ldots$

For example, the search may continue for up to 30 decoding steps.

Candidates can therefore have different lengths.

Each completed candidate is evaluated using the normalized score:

$$\text{Score}(y) = \frac{1}{T_y^{\alpha}} \sum_{t=1}^{T_y} \log P(y_t | y_{<t}, x)$$

The final output is the candidate with the largest score:

$$\boxed{y^* = \arg\max_y \frac{1}{T_y^{\alpha}} \sum_{t=1}^{T_y} \log P(y_t | y_{<t}, x)}$$

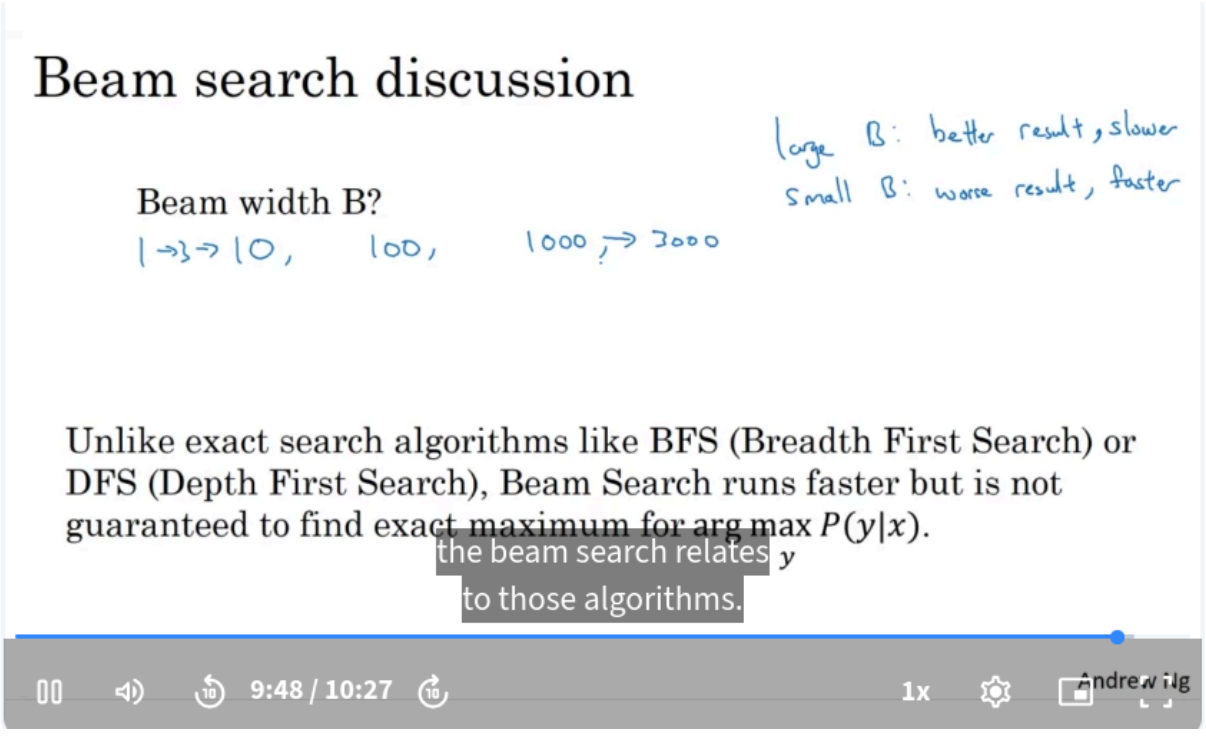
Why Use Log Probabilities?

Instead of multiplying probabilities: $P(y|x) = \prod_t P(y_t | y_{<t}, x)$

we use: $\log P(y|x) = \sum_t \log P(y_t | y_{<t}, x)$

This is useful because:

- Multiplying many small probabilities can cause numerical underflow.
- Summation is computationally easier than multiplying many tiny numbers.
- Maximizing (P(y|x)) is equivalent to maximizing (\log P(y|x)).



Choosing the Beam Width

The **beam width** B controls how many candidate sequences Beam Search keeps at each decoding step.

There is a trade-off between:

- Search quality
- Computation time
- Memory usage

Large Beam Width

If B is large:

- The algorithm considers more candidate sequences.
- It is more likely to find a better output.
- Search quality usually improves.
- However:
  - Computation becomes slower.
  - Memory usage increases.

So: Large B⇒better search, higher cost

Small Beam Width

If B is small:

- Fewer candidate sequences are kept.
- Search is faster.
- Memory usage is lower.
- But promising sequences may be discarded too early.

So: Small B⇒faster search, potentially worse result

Typical Beam Width Values

Some rough examples:

- B=1: equivalent to **Greedy Search**
- B=3: relatively small
- B≈10: common in some production systems

- B≈100: relatively large
- B=1000 or 3000: sometimes used in research when trying to maximize performance

The best value depends heavily on the application.

Diminishing Returns

Increasing B usually helps most when moving from very small values.

For example:

$B=1 \rightarrow 3 \rightarrow 10$

may produce significant improvements.

But increasing from:

$B=1000 \rightarrow 3000$

may produce only a small improvement while requiring much more computation.

This is known as **diminishing returns**.

Beam Search Is Approximate

Beam Search is an **approximate search algorithm**.

It tries to find: $y^* = \arg\max_y P(y|x)$

but it is **not guaranteed** to find the true global maximum.

Why?

Because at every step, it discards all but the best B partial sequences.

A discarded sequence might later have become the globally best sequence.

Beam Search vs. Exact Search

Algorithms such as **Breadth-First Search (BFS)** or **Depth-First Search (DFS)** can systematically explore a search space.

Beam Search instead aggressively removes low-scoring candidates.

Conceptually:

Method	Search Property
Greedy Search	Keeps 1 candidate
Beam Search	Keeps B candidates
Exhaustive/Exact Search	Explores all required possibilities

Beam Search is much more practical for sequence generation because the complete search space is enormous.

How to Choose B

Beam width should be treated as a **hyperparameter**.

A practical approach is to try several values, such as:

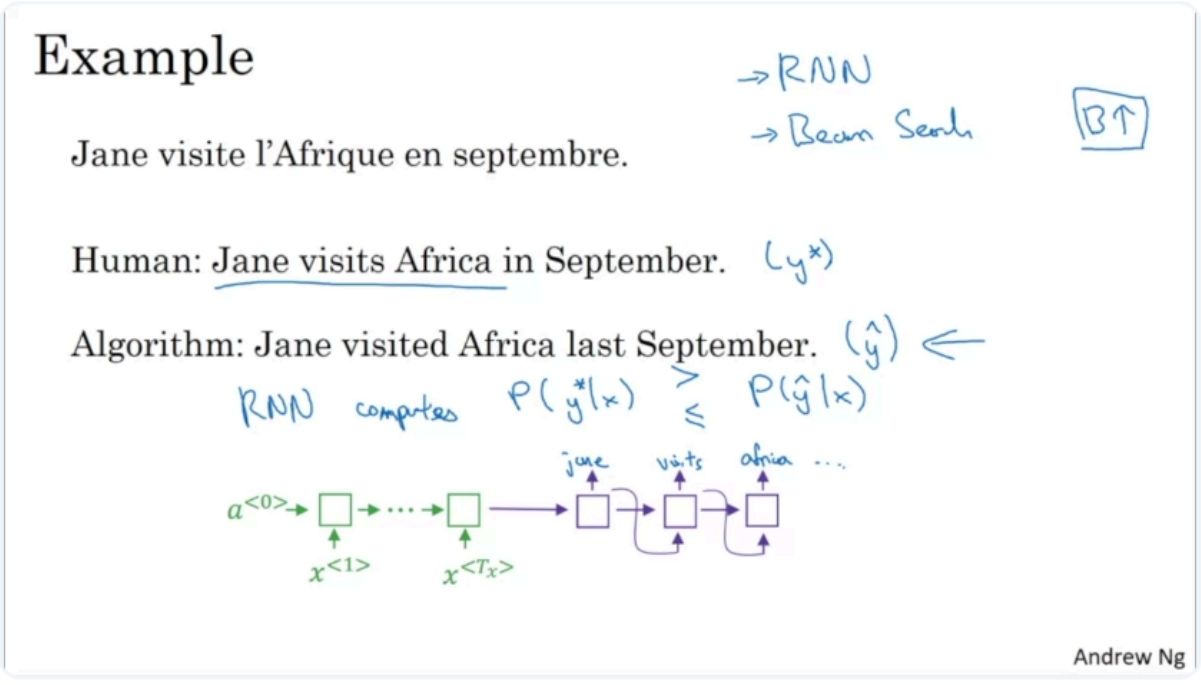
$B \in \{1,3,5,10,20,...\}$

and evaluate the trade-off between:

- Output quality
- Inference speed
- Available memory

Increasing B indefinitely is usually not worthwhile because of diminishing returns.

Error Analysis in Beam Search



Error Analysis for Beam Search

- Beam Search is an **approximate / heuristic search algorithm**.
- Because it only keeps the top B candidates, it is not guaranteed to return the most likely sentence.
- When the final translation is bad, the error may come from either:
  - The **RNN / encoder-decoder model**
  - The **Beam Search algorithm**

The goal of error analysis is to determine which component is responsible.

Example

Suppose the input French sentence is:

> Jane visite l'Afrique en septembre.

The human reference translation is:

y∗="Jane visits Africa in September"

But Beam Search outputs:

y^="Jane visited Africa last September"

The second translation changes the meaning, so it is clearly worse.

Two Components of the System

The translation system has two main parts:

Neural Network Model

The encoder-decoder model estimates:

$$P(y|x)$$

It determines how good a particular translation looks according to the learned model.

Beam Search

Beam Search tries to approximately solve:

$$\arg\max_{y} P(y|x)$$

It searches through possible translations using the probabilities produced by the neural network.

The Key Error Analysis Test

Compute two probabilities using the neural network:

$$P(y^*|x)$$

and

$$P(\hat{y}|x)$$

where:

- y^*: human reference translation
- $\hat{y}$: translation returned by Beam Search

Then compare them.

Case 1: Beam Search Is the Problem

Suppose:

$$\boxed{P(y^*|x) > P(\hat{y}|x)}$$

The neural network actually believes the human translation is better.

But Beam Search returned $\hat{y}$.

Therefore:

$$\boxed{\text{Search error}}$$

The model knows a better translation exists, but Beam Search failed to find it.

Possible improvements include:

- Increase beam width B
- Improve the search procedure
- Tune Beam Search hyperparameters

Case 2: The Model Is the Problem

Suppose:

$$\boxed{P(y^*|x) \leq P(\hat{y}|x)}$$

The neural network itself assigns a higher probability to the bad translation.

Even a perfect search algorithm would therefore prefer $\hat{y}$.

So:

$$\boxed{\text{Modeling error}}$$

The problem is not mainly Beam Search.

Instead, effort should go into improving the sequence-to-sequence model, for example:

- Better training data
- Better architecture
- Better optimization
- Better modeling of the translation task

Why Increasing Beam Width Is Not Always the Answer

It may seem natural to always increase B.

But if:

$$P(y^*|x) < P(\hat{y}|x)$$

then improving the search will not fix the fundamental problem.

A better search algorithm may simply become even better at finding the wrong translation preferred by the model.

So first determine whether the problem is:

$$\boxed{\text{Search}}$$

or:

$$\boxed{\text{Model}}$$

before deciding where to spend effort.

Error Diagnosis Rule

For each bad Beam Search output:

$$\boxed{P(y^*|x) > P(\hat{y}|x) \Rightarrow \text{Beam Search problem}}$$

$$\boxed{P(y^*|x) \leq P(\hat{y}|x) \Rightarrow \text{RNN / model problem}}$$

This is the central idea behind Beam Search error analysis.

Intuition

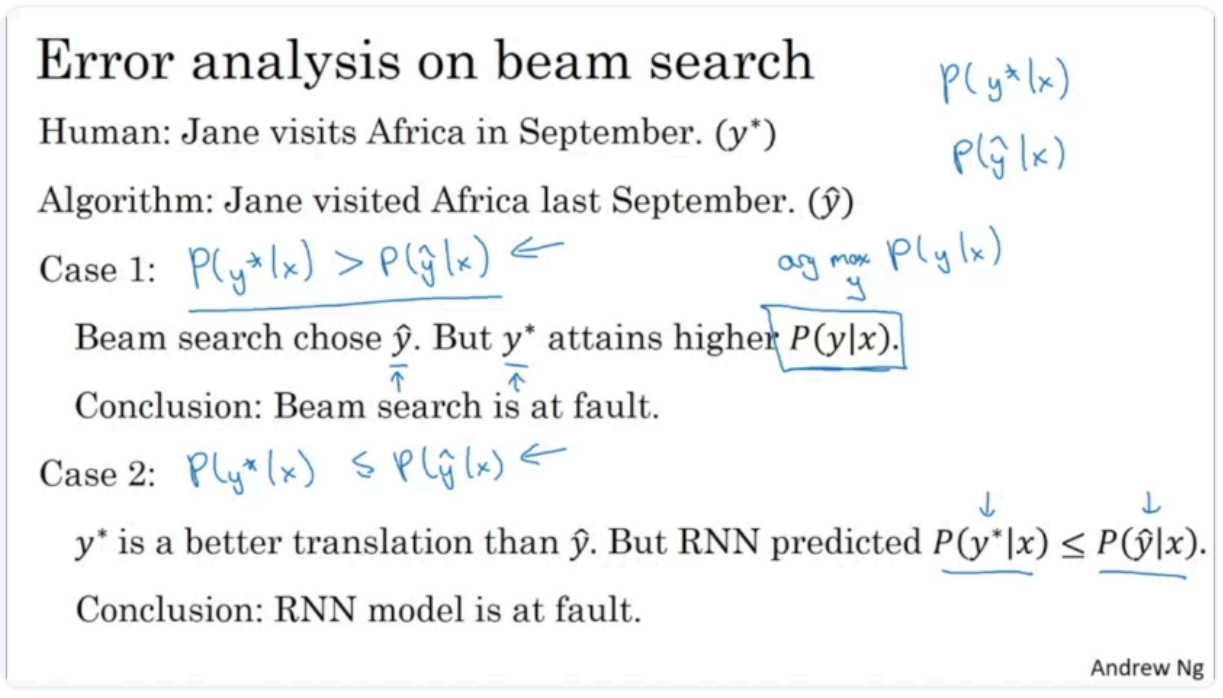
The neural network and Beam Search have different jobs:

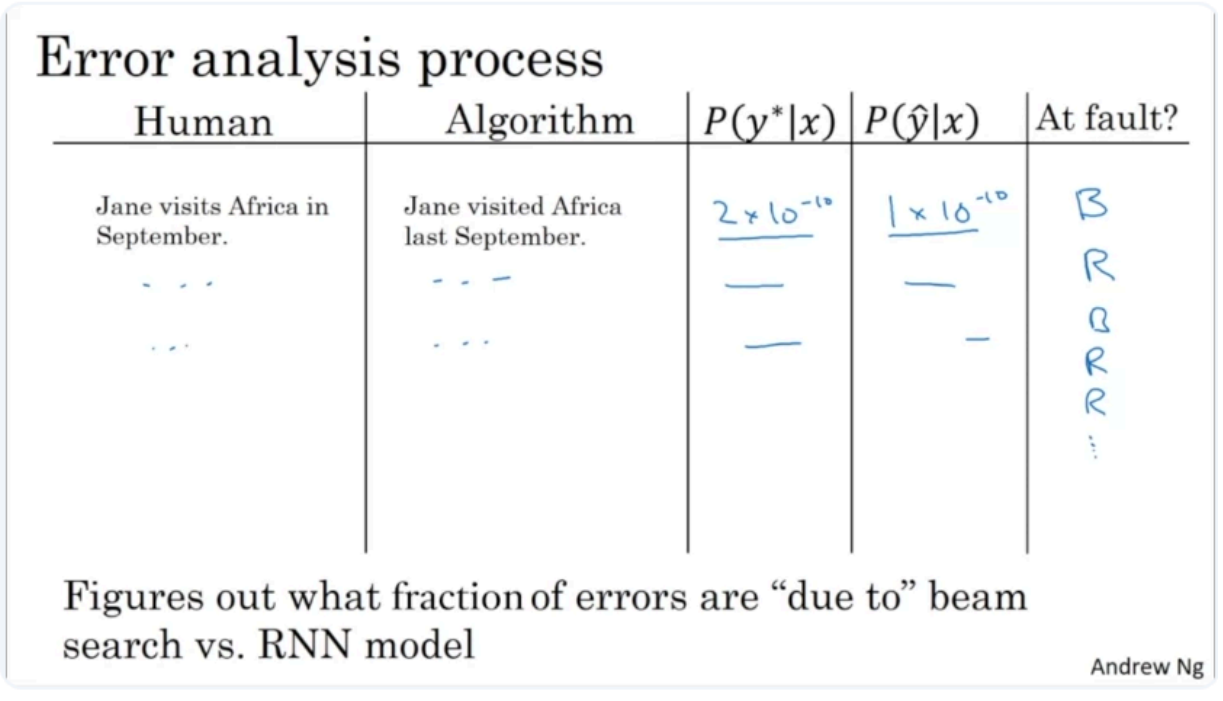
$$\text{Neural Network} \rightarrow \text{assign probabilities}$$

$$\text{Beam Search} \rightarrow \text{find high-probability sequences}$$

So if the model assigns the correct translation a high score but Beam Search misses it, improve **search**.

If the model itself prefers the wrong translation, improve the **model**.



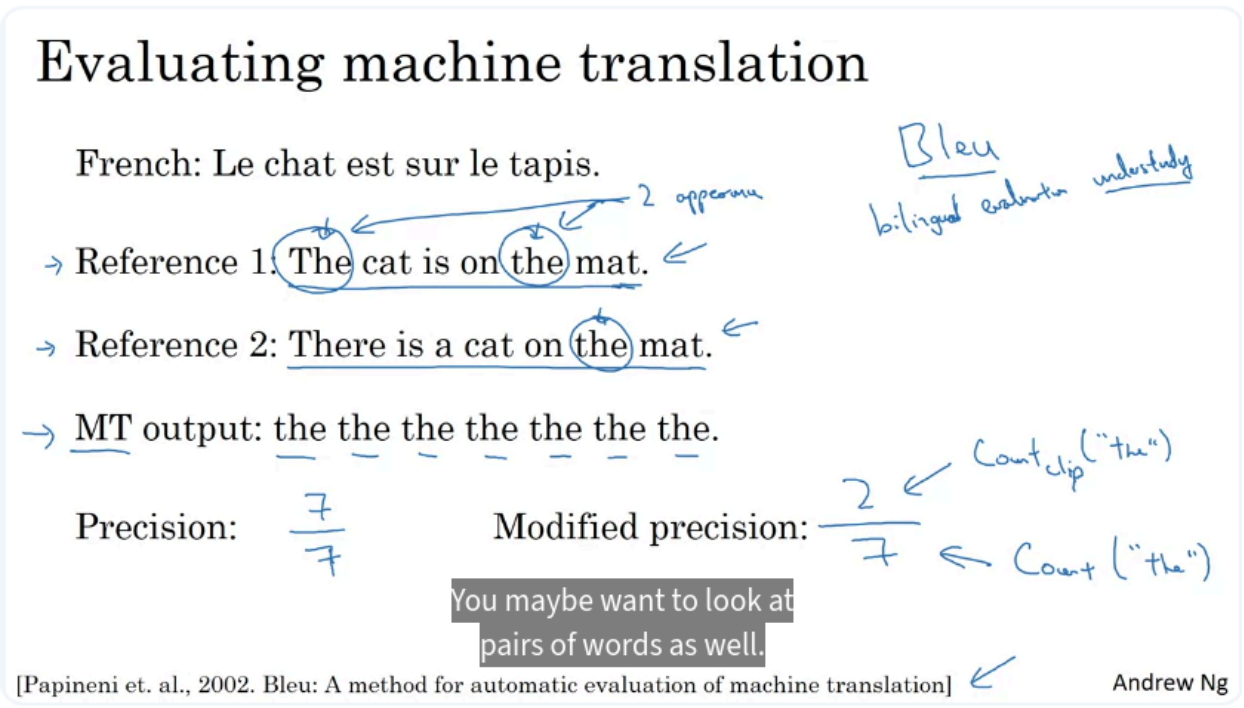


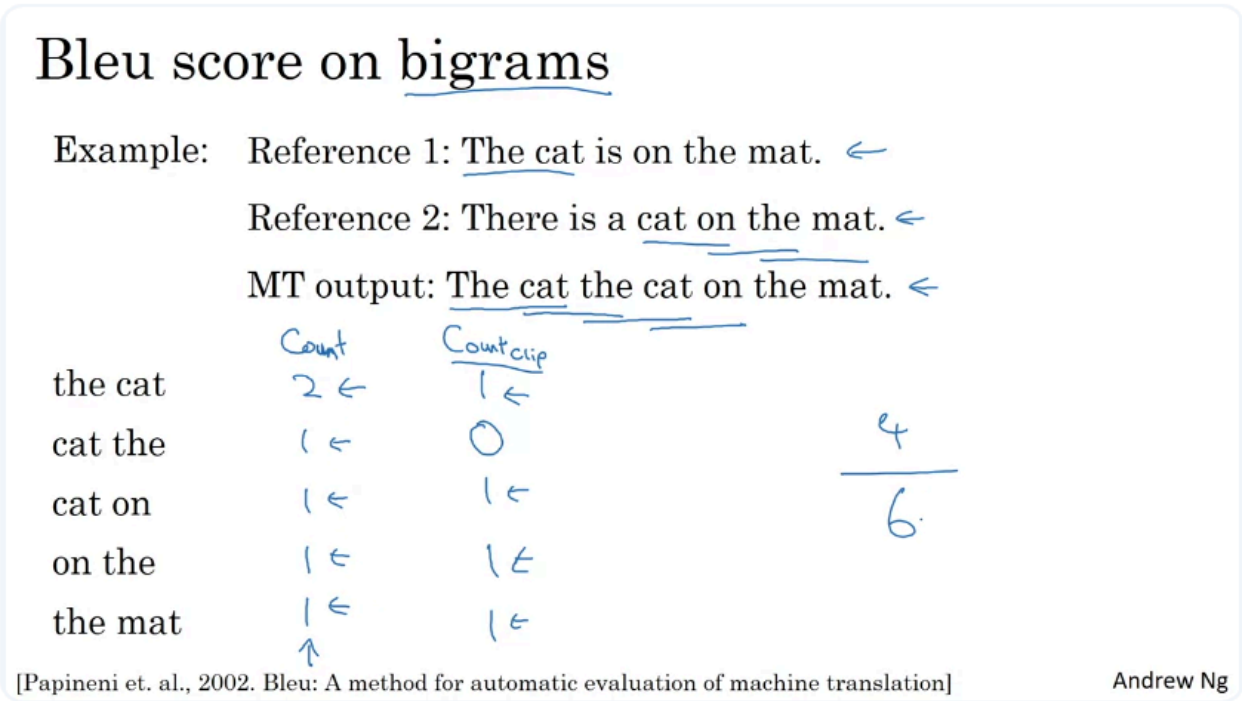
B: Beam Search

R: RNN model

BLEU Score

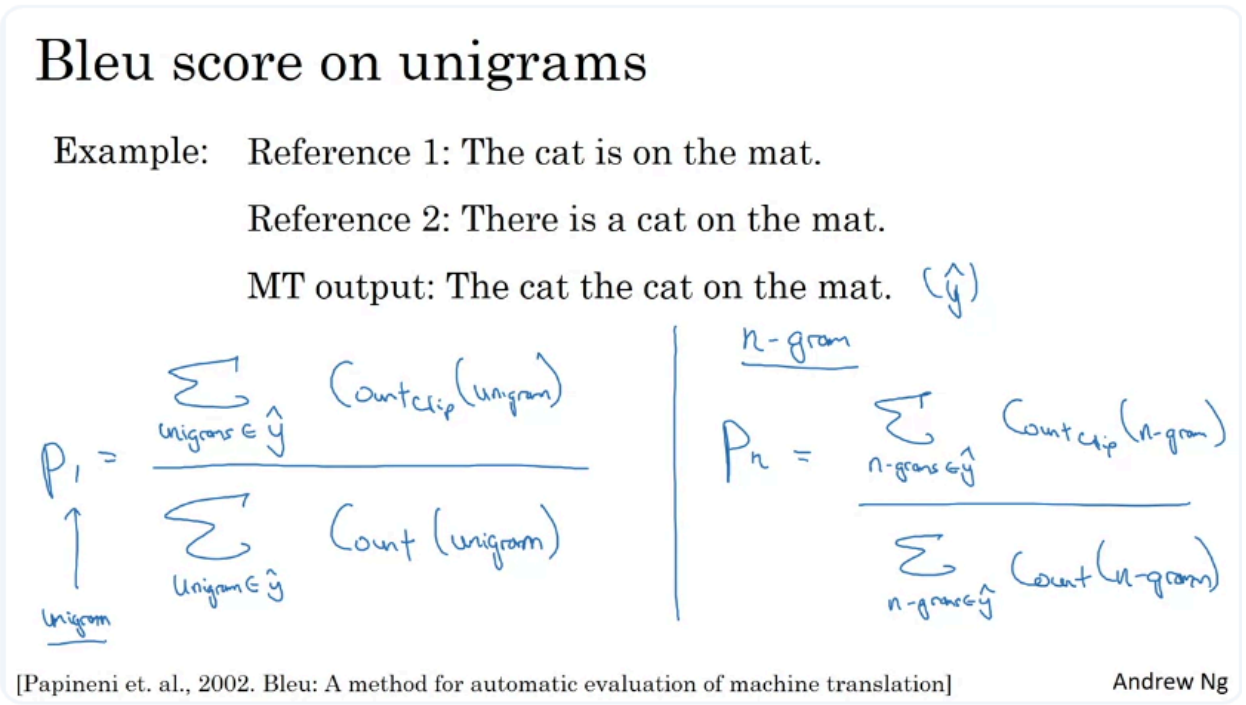
Bleu: Bilingual Evaluation Understudy

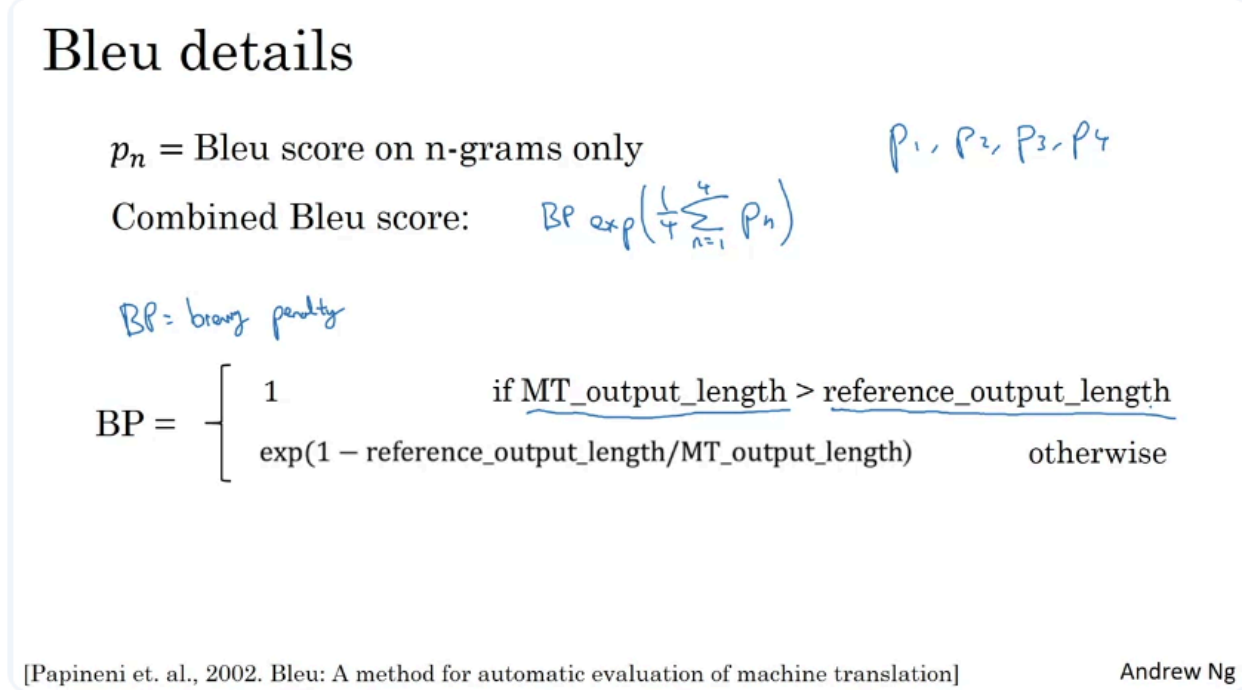




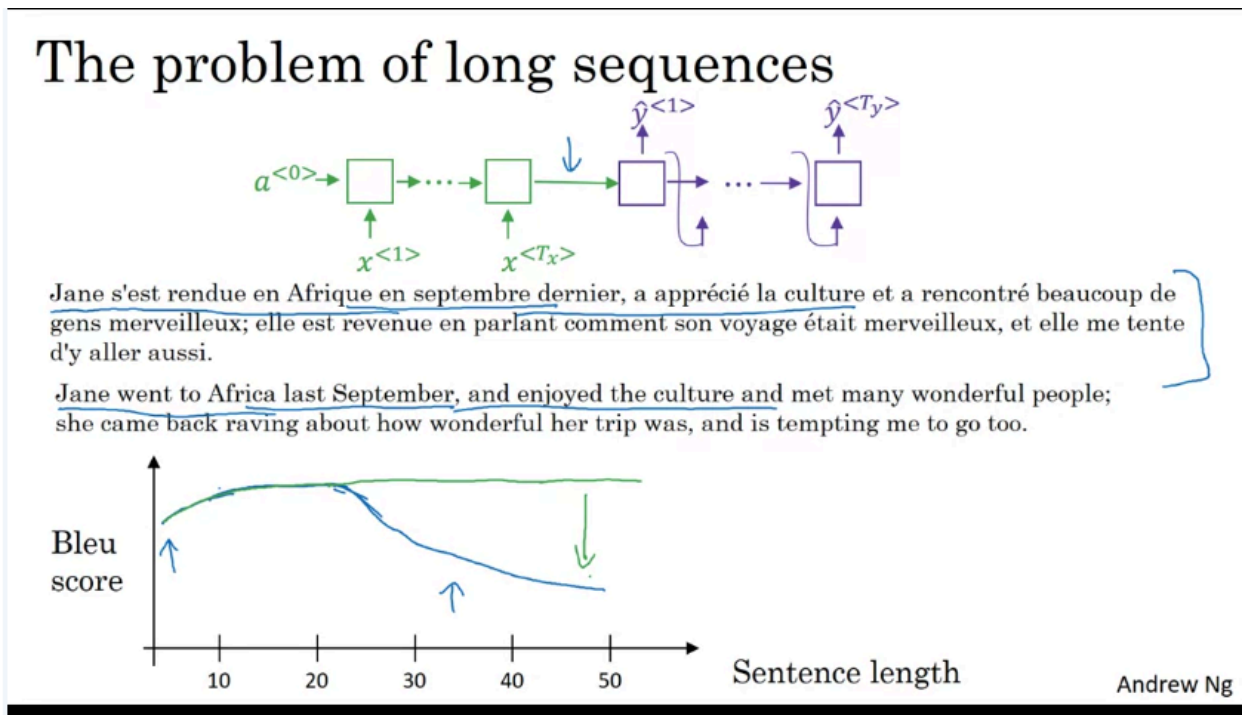
- Count: the n-gram words appear in the Machine Translation output (MT output)
- Count clip: the count of n-gram words appear in the reference translation

Modified Precision





Attention Model Intuition

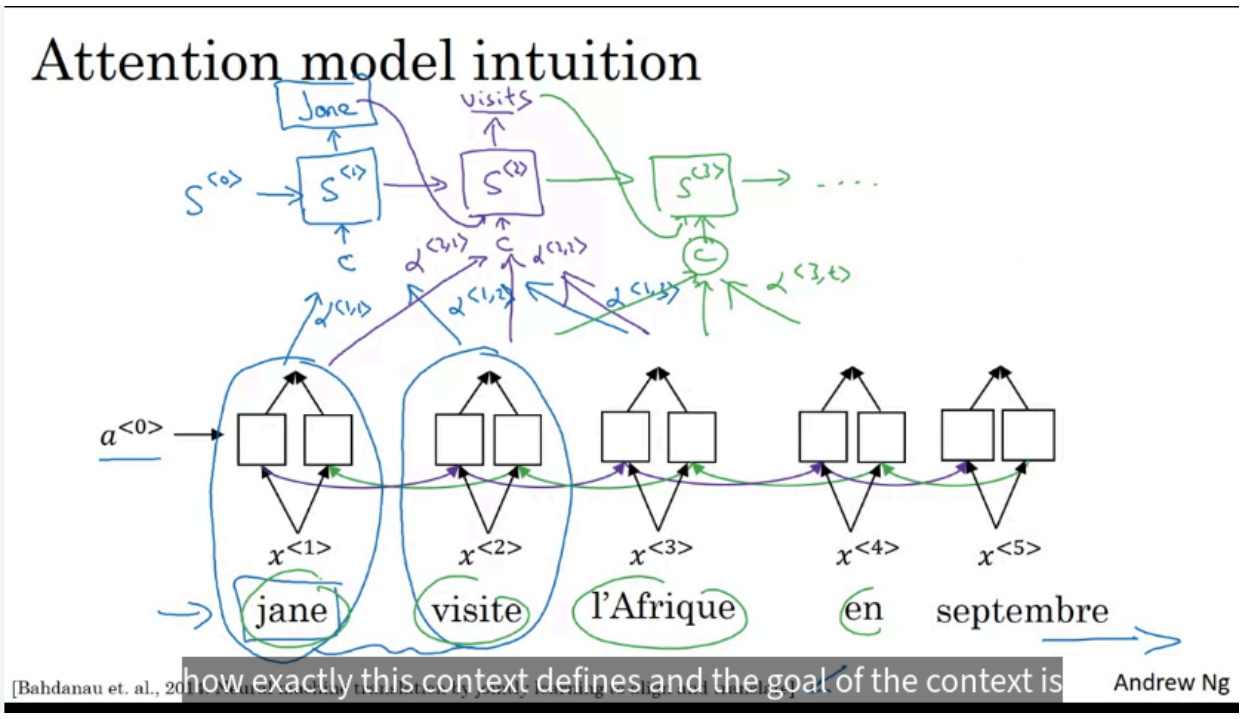


The problem of previous machine translation models:

- Cannot remember the long sentence (the performance is the blue line)
- This leads to wrong translation

Attention:

- Help maintain the performance



Attention Model: Motivation

- The standard **Encoder–Decoder architecture** uses one RNN to read the entire input sentence and another RNN to generate the output sentence.
- The encoder compresses the whole input into a fixed representation that the decoder must rely on.
- This works reasonably well for short sentences, but performance drops for long sentences because the model has to effectively **remember too much information at once**.

Why Long Sentences Are Difficult

A human translator usually does not:

> Read a very long sentence, memorize everything, then translate it from scratch.

Instead, a human tends to:

- Read part of the sentence.
- Translate that part.
- Look at the next relevant part.
- Continue step by step.

The **Attention Model** follows a similar idea.

Main Idea of Attention

- Instead of forcing the decoder to depend on one fixed representation of the entire input, attention lets the decoder **focus on different parts of the input sentence at each output step**.
- When generating each output word, the model determines which input words are most relevant.

So:

Each output step attends to relevant input positions

Encoder with a Bidirectional RNN

Suppose the input French sentence is:

> Jane visite l'Afrique en Septembre.

A **Bidirectional RNN** processes the sentence and produces a representation for each input position.

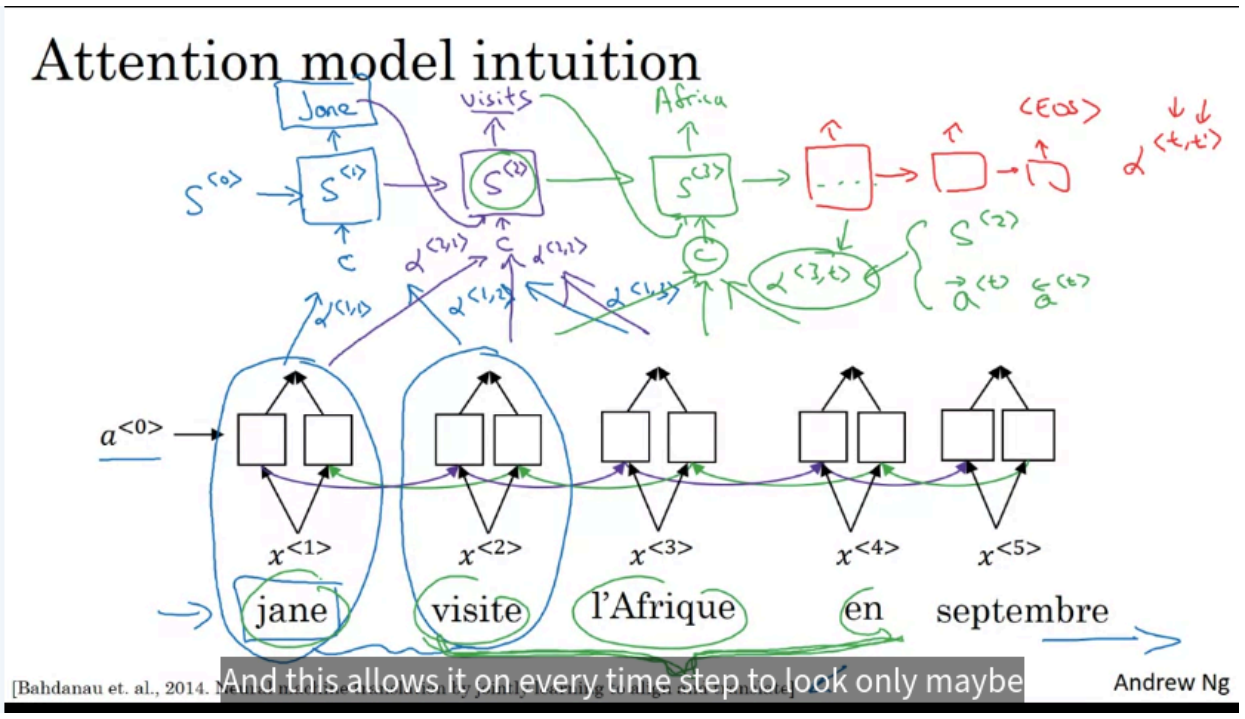
For each input word t′, we obtain an activation:

a⟨t′⟩

which contains information from both:

- Words before that position.
- Words after that position.

Therefore, every input position has a rich contextual representation.



Decoder Hidden State

A separate RNN generates the English translation:

> Jane visits Africa in September.

The decoder hidden states are denoted:

s⟨1⟩,s⟨2⟩,s⟨3⟩,...

instead of a, to distinguish them from the encoder activations.

Attention Weights

When generating output word t, the model computes attention weights over all input positions.

The attention weight is written as:

α⟨t,t′⟩

It means:

> When generating output word t, how much attention should the decoder pay to input word t′?

For example:

α⟨1,1⟩

describes how much the decoder should focus on the first French word when generating the first English word.

Example: Generating "Jane"

When producing:

> Jane

the model may assign high attention to the first input word:

> Jane

and much lower attention to words near the end of the sentence.

Conceptually:

$\alpha\langle 1,1\rangle \gg \alpha\langle 1,5\rangle$

The model focuses mainly on the relevant part of the input.

Context Vector

The attention weights are used to construct a **context vector**:

$$c^{\langle t\rangle}$$

The context vector summarizes the input information that is most relevant for generating output word t.

Conceptually:

$$c^{\langle t\rangle} = \sum_{t'} \alpha^{\langle t,t'\rangle} a^{\langle t'\rangle}$$

So input positions with larger attention weights contribute more strongly.

Attention Changes at Every Output Step

When generating the second word:

> visits

the model computes a new set of attention weights:

$$\alpha^{\langle 2,1\rangle}, \alpha^{\langle 2,2\rangle}, \ldots$$

It may now focus mostly on:

> visite

When generating:

> Africa

it computes another set:

$$\alpha^{\langle 3,t'\rangle}$$

and may focus strongly on:

> l'Afrique

Therefore:

$$\boxed{\text{Attention weights depend on the current decoder step}}$$

What Determines Attention?

The attention weight for output step t and input position t' depends on information such as:

- The encoder representation:

$$a^{\langle t'\rangle}$$

- The previous decoder state:

$$s^{\langle t-1\rangle}$$

These tell the model:

> Given what I have already translated, which part of the source sentence should I focus on next?

Decoder with Attention

At every output step, the decoder receives:

- Previous decoder state.
- Previous generated word.
- Current attention-based context vector.

Conceptually:

$$(s^{\langle t-1\rangle}, y^{\langle t-1\rangle}, c^{\langle t\rangle}) \rightarrow s^{\langle t\rangle} \rightarrow y^{\langle t\rangle}$$

This continues until the model generates:

$$<\text{EOS}>$$

Attention vs. Basic Encoder–Decoder

Basic Encoder–Decoder	Attention Model
Compresses input into one fixed representation	Keeps representations for all input positions
Decoder relies on the same encoded information	Decoder dynamically chooses relevant information
Difficult for long sentences	Handles long sentences better
Requires strong memorization	Focuses on relevant input regions

Why Attention Helps

Attention reduces the need for the encoder to compress every detail of a long sentence into one fixed vector.

Instead:

$$\text{Long input} \rightarrow \text{many contextual encoder states}$$

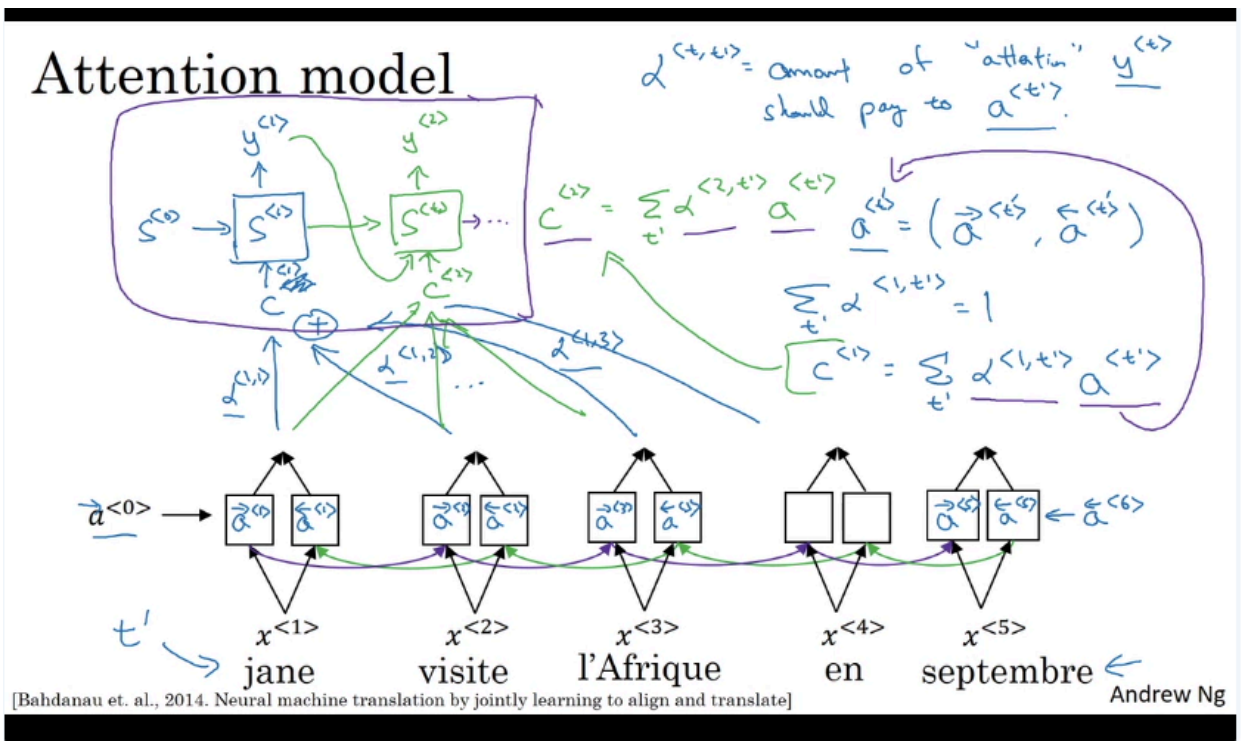
and at each decoding step:

$$\text{Attention} \rightarrow \text{select relevant information}$$

This is why attention can significantly improve translation quality, particularly for longer sentences.

Attention Model

At time 5:32, Andrew says "This network up here looks like a pretty standard RNN sequence with the context vectors as output". The correct wording should say that the context vectors are "**inputs**" to the post-attention RNN.

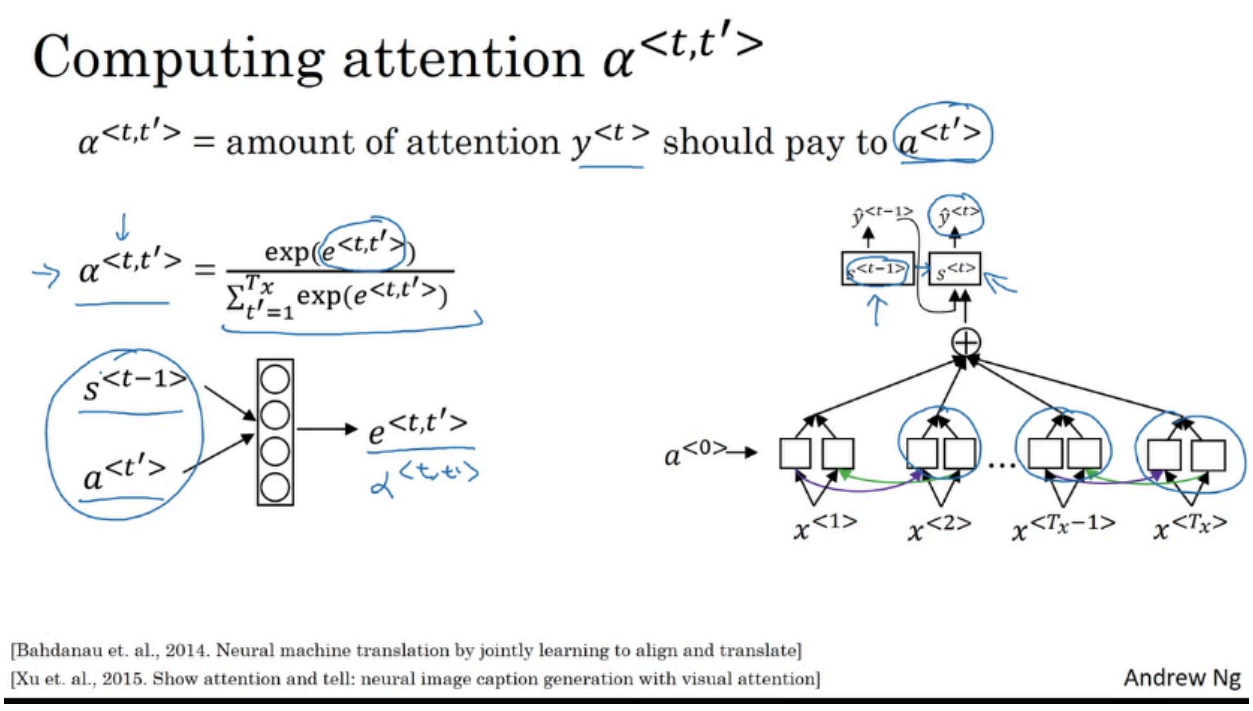


The input contex of the cell i is computed by the formula:

$$c^{\langle i\rangle} = \sum_{t'} \alpha^{\langle i,t'\rangle} a^{\langle t'\rangle}$$

Where:

- t' is the position of input words
- α: weight (how much we should pay attention to every other words)



Computational Cost

If:

- Input length = Tx
- Output length = Ty

then attention computes approximately:

Tx×Ty

attention scores.

So the computational cost is roughly:

O(TxTy)

This can be expensive for very long sequences, although it is usually manageable for normal machine translation sentence lengths.

Attention Beyond Machine Translation

The same attention idea can be applied to other tasks.

For example:

Image Captioning

- Instead of attending to different input words, the model attends to different **regions of an image**.
- While generating each caption word, it focuses on the image region most relevant to that word.

Date Normalization

Attention can also be used in sequence-to-sequence tasks such as converting:

> 3 May 1979

into a normalized form like:

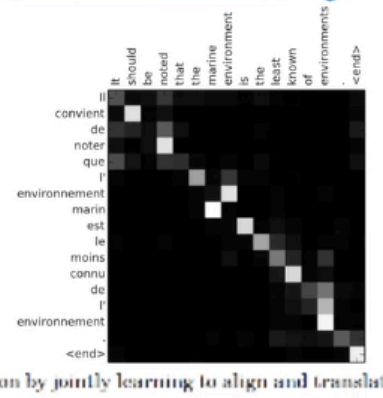
> 1979-05-03

Attention examples

July 20th 1969 ⟶ 1969 – 07 – 20

23 April, 1564 ⟶ 1564 – 04 – 23

Visualization of $\alpha^{<t,t'>}$:



[Bahdanau et. al., 2014. Neural machine translation by jointly learning to align and translate]

Andrew Ng